🛡️ NOIR: Technical Specifications, Dataset Credits & Training Dossier

This document is a comprehensive, production-grade technical dossier for **NOIR** (Autonomous Cross-Repository Code Repair & Quality Intelligence Engine). It details all models, exact hyperparameters, dataset sources with complete author attributions, training infrastructure, file sizes, and architecture specifications for evaluation, presentation, and academic/engineering review.

---

1. 🧠 Foundation Model & Quantization Specifications

1.1 Base Architecture

- **Foundation Model**: `Qwen2` — `Qwen/Qwen2.5-Coder-7B-Instruct`
- **Model Creator / Origin**: Alibaba Cloud Qwen Team (Open Weights Apache 2.0 / Tongyi Qianwen License)
- **Total Parameters**: **7.61 Billion** (7,615,616,512 parameters)
- **Architecture Type**: Decoder-only causal language model with SwiGLU activations and RoPE (Rotary Position Embeddings)
- **Native Context Window**: **32,768 tokens**
- **Active NOIR Inference Context**: **8,192 tokens** (`num_ctx 8192`)
- **Hidden / Embedding Dimension**: **3,584**
- **Number of Attention Heads**: **28**
- **Number of Key-Value Heads (GQA)**: **4** (Grouped-Query Attention for high memory efficiency)
- **Vocabulary Size**: **152,064 tokens**

1.2 Quantization & Export Pipeline

- **Base Training Quantization**: **4-bit NormalFloat (NF4)** via `bitsandbytes` (`unsloth/Qwen2.5-Coder-7B-Instruct-bnb-4bit`)
- **Quantization Converter**: `llama.cpp` (`convert_hf_to_gguf.py`)
- **Quantization Scheme**: `Q4_K_M` (4-bit Medium K-Quantization: critical attention and feed-forward projection layers retain higher precision while embedding and remaining weights use 4-bit)
- **Raw Checkpoints & Unmerged Artifacts Archive**: **~12.0 GB**
- **Final Quantized GGUF File**: `qwen2.5-coder-7b-instruct.Q4_K_M.gguf`
- **GGUF File Size on Disk**: **4.68 GB** (~4.7 GB)
- **Ollama Registered Model**: `noir:latest` (`ID: 3778b44e78e5`)
- **Local Hardware Acceleration**: **79% GPU Offload** (27 of 29 transformer blocks resident in GPU VRAM, 2 on system RAM)
- **Average Inference Latency**: **< 2.5 seconds** per code file

---

2. 📚 Training Datasets & Open-Source Credits

The fine-tuning dataset consists of **511 high-quality curated code repair pairs** (`backend/dataset/qlora_finetune_data.jsonl`, **358.6 KB**), combining an accredited open-source benchmark dataset with synthetic architectural augmentation:

2.1 Primary Open-Source Foundation: Hugging Face CodeAlpaca

- **Dataset Name**: `CodeAlpaca-20k`
- **Hugging Face Hub Repository**: `sahil2801/CodeAlpaca-20k`
- **Original Creator / Attribution**: Sahil Chaudhary (@sahil2801)
- **License**: Creative Commons Attribution-NonCommercial 4.0 (CC BY-NC 4.0)
- **Filtering & Extraction Strategy**:
- Filtered specifically for instructions containing keywords: `fix`, `refactor`, `bug`, `optimize`, `error`, `improve`.
- Extracted paired `input` (buggy/vulnerable code snippet) and `output` (corrected, clean implementation).
- Mapped into NOIR's cross-repository prompt format via `backend/scripts/finetuning/build_dataset_huggingface.py`.

2.2 Secondary Synthetic Augmentation: Google Gemini

- **Model Used**: `gemini-3.8-flash` via Google GenAI SDK (`google-genai`)
- **Generator Script**: `backend/scripts/finetuning/build_dataset_with_gemini.py`
- **Purpose**: Injected specialized cross-repository architectural context into each example, teaching the model to resolve:
- Dangerous dynamic code execution (`eval()`, `exec()`, raw `os.system()`).
- Silent exception swallowing (bare `except:` and `except Exception: pass`).
- Unhandled zero-division and missing arithmetic guards.
- Deep monolithic nesting flattened into early-return guard clauses.
- XSS injection hazards (`dangerouslySetInnerHTML`).

2.3 Dataset Schema (ChatML / Qwen Token Format)

Each sample in `qlora_finetune_data.jsonl` adheres to:

```
{
  "instruction": "You are NOIR, an autonomous cross-repository code repair engine. Fix the bugs and arc
  "context": "Architectural Insight: Use the function to debug the given program and prevent the segmer
  "input": "int main() { ... }",
  "output": "```python\nint main() { ... }\n```",
  "file_path": "backend/service.py",
  "timestamp": "2026-09-17T19:50:00.000Z"
}
```

During training, these are formatted into Qwen 2.5's native chat template:

```
<|im_start|>user
{instruction}

CROSS-REPOSITORY INSIGHTS:
{context}

Code:
```{input}```<|im_end|>
<|im_start|>assistant
{output}<|im_end|>
```

3. ⚙️ QLoRA Fine-Tuning Methodology & Hyperparameters

- **Fine-Tuning Framework**: **Unsloth** (`unslothai/unsloth`) + Hugging Face **TRL** (`SFTTrainer`) + **PEFT** + **bitsandbytes**
- **Compute Infrastructure**: Google Colab with NVIDIA GPU (Tesla T4 / A100)
- **Training Paradigm**: **QLoRA** (Quantized Low-Rank Adaptation)
- **Quantization Precision**: 4-bit NormalFloat (`load_in_4bit=True` , `bnb_4bit_quant_type="nf4"`)
- **Compute Precision**: `bfloat16` (with `fp16` fallback)

Exact Hyperparameters:

Hyperparameter	Value	Description / Purpose
LoRA Rank (r)	`16`	Rank of the low-rank adaptation decomposition matrices
LoRA Alpha (α)	`16`	Scaling factor ($\alpha / r = 1.0$)
LoRA Dropout	`0`	Unsloth optimized 0 dropout for maximum training speed
Target Modules	`7 Modules`	`q_proj` , `k_proj` , `v_proj` , `o_proj` , `gate_proj` , `up_proj` , `down_proj`
Trainable Parameters	**~20.1 Million**	**~0.26%** of total weights (prevents catastrophic forgetting)
Per-Device Batch Size	`2`	Number of samples processed per GPU pass
Gradient Accumulation	`4`	Effective batch size of $2 \times 4 = 8$
Total Max Steps	`120`	~2 full training epochs over the 511 curated pairs
Learning Rate	`2e-4`	Peak learning rate
LR Scheduler	`linear`	Linear decay schedule
Warmup Steps	`5`	Initial learning rate warmup steps
Optimizer	`adamw_8bit`	Memory-efficient 8-bit AdamW
Weight Decay	`0.01`	L2 regularization
Random Seed	`3407`	Fixed seed for complete experiment reproducibility
Gradient Checkpointing	`unsloth`	Custom low-memory gradient checkpointing
```

4. 🔍 Cross-Repository Structural RAG Engine

NOIR repairs code based on real architectural benchmarks harvested from high-star open-source peers:

4.1 Fingerprint Extraction (`backend/noir_fingerprint.py`)

- **AST Parsing**: Analyzes Python/JS Abstract Syntax Trees to extract functions, classes, decorators, and imports.
- **Dependency Profiling**: Parses `requirements.txt`, `package.json`, `pyproject.toml`, `go.mod`, `Cargo.toml`.
- **Structural Vectorizer**: Converts AST and directory profiles into **512-dimensional dense structural vectors** via `scikit-learn` `HashingVectorizer`.

4.2 SQLite Vector Store & Strict Session Isolation

- **Storage Engine**: Embedded SQLite database (`.noir/vector_store.sqlite3`).
- **Isolation Guarantee**: Explicit `conn.commit()` on table wipe (`DELETE FROM repo_vectors`) + `allowed_repo_ids` boundary filtering. Guarantees **0% cross-pollination** between different audited repositories.
- **Hybrid Similarity Metric**: $$\text{Score} = (0.70 \times \text{Cosine Semantic Similarity}) + (0.30 \times \text{Jaccard Structural AST Similarity})$$

4.3 GitHub Domain Harvester (`backend/noir_cross_repo.py`)

- **Semantic Domain Extraction**: Extracts functional nouns from repo identifiers and descriptions while filtering generic noise words.
- **Catalog Blacklist**: Automatically strips generic aggregate repositories (`awesome-*`, `free-programming-books`, `public-apis`, `interview`).
- **Cache-Busted Downloader**: Queries `repo.get_commits()[0].sha` to bypass GitHub's Akamai CDN 15-minute zipball cache, ensuring freshly merged PR commits are audited immediately.

---

5. 🛡️ Objective 6-Dimension Quality Scorecard (`backend/noir_quality.py`)

Calculates a deterministic **0–100 Health Index** and letter grade:

Dimension	Weight	Detection Criteria
1. Security & Vulnerabilities	**25%**	Scans for `eval()`, `exec()`, hardcoded secrets/tokens, and unquoted shell calls.
2. Maintainability & Clean Code	**20%**	Excessive nesting depth (≥ 4 levels) and monolithic files (> 400 lines).
3. Error Handling & Resilience	**20%**	Python `ast.Div` checks, bare `except:` blocks, and unprotected `JSON.parse`.
4. Architecture & Modularity	**15%**	Layer separation, component cohesion, and directory hierarchy.
5. Test & CI/CD Readiness	**10%**	Presence of automated tests (`tests/`) and CI pipelines (`.github/workflows`).
6. Documentation & Standards	**10%**	Production README coverage, API routes documentation, and usage instructions.

Letter Grades: - **A+ (90–100)**: Production Ready - **A (80–89)**: Robust Architecture - **B (70–79)**: Needs Architecture Polish - **C (60–69)**: Moderate Technical Debt - **D (< 60)**: Critical Refactoring Required

---

6. 💻 Full Software Stack & Tool Attribution

Category	Technology	Version / Spec	Primary Role
LLM Runtime	**Ollama**	`v0.5.x`	Local execution of GGUF model with GPU offload
Base Model	**Qwen 2.5 Coder 7B**	`7.6B params`	Base reasoning and code intelligence foundation
Model Format	**GGUF (llama.cpp)**	`Q4_K_M (4.68 GB)`	High-efficiency 4-bit medium quantization
Fine-Tuning Stack	**Unsloth**	Latest Colab	$2\times$ faster QLoRA training with 70% less VRAM
Training API	**TRL (SFTTrainer)**	`trl < 0.9.0`	Supervised fine-tuning trainer
Quantization Lib	**bitsandbytes**	4-bit NF4	Low-bit parameter quantization
Base Dataset	**CodeAlpaca-20k**	`sahil2801` (HF)	Open-source code repair training benchmark
Data Augmentation	**Gemini 3.8 Flash**	Google GenAI	Synthetic cross-repo context generation
Backend Framework	**FastAPI**	`v0.115+`	Async REST API orchestration (`localhost:8000`)
Server Engine	**Uvicorn**	`v0.32+`	ASGI high-performance application server
Git Orchestration	**PyGithub**	`v2.5+`	GitHub REST API v3: OAuth, tree, commit, PR creation
Vector Store	**SQLite 3**	Embedded	In-process relational vector database
Frontend UI	**React 18**	`v18.3.1`	Single-page reactive dashboard
Build Engine	**Vite**	`v8.3.0`	Production bundler (1.18s compilation)
Styling	**Tailwind CSS**	`v3.4+`	Glassmorphic dark-mode developer studio design
Iconography	**Lucide React**	`v0.460+`	UI icon kit
Auth Protocol	**GitHub OAuth 2.0**	`repo, user`	Secure developer credential delegation